

VANADIS 3D - introducing non-flat terrain and velocity fields

Practical guide for the current terrain-following core (state: 12 Sep 2026)

Vanadis 3D - Marek Chodorski

Key answer: Yes - VANADIS can use a measured, CFD-derived or meteorological velocity field $v(x,y,z,t)$. If the field already represents the real flow over the same terrain and already respects the intended ground condition, do not automatically add the terrain-consistent slope correction.

1. Terrain convention used by VANADIS

In VANADIS, **x is the vertical coordinate, while y and z are horizontal**. Therefore the ground surface is written as $x = xg(y,z)$. This is different from the common convention in which z is vertical.

Quantity	Meaning in the current code
x	vertical coordinate [m]
y, z	horizontal coordinates [m]
xground(jy,jz)	ground elevation xg at node (dy(jy), dz(jz)) [m]
dx(:)	reference vertical levels; currently 0, 20, 35, 85, 135, 230, 420 m
xbase = dx(1)	reference lower level, currently 0 m
xtop = dx(nax)	fixed top boundary, currently 420 m

The current geometry initialization sequence is:

```
call wsp_na_o(..., dx, dy, dz)
call terrain_init(dy, dz, nay, naz)
call siatka(xx, yy, zz, dx, dy, dz, nax, nay, naz)
```

The key architectural rule is that the same xground array is used by both the mesh generator and terrain_geometry, while terrain_geometry is also used by velocity_field. Thus the ground geometry and any terrain-based velocity correction are based on one consistent surface definition.

2. How to introduce a new non-flat terrain

2.1. Simplest method - fill xground(jy,jz)

In the current source, terrain_init first sets the terrain to zero and then inserts the verified 0,1,4,7 m test patch at nodes 10/11. To prescribe another terrain, replace this block with your own xground(jy,jz) elevations. The routines siatka, terrain_geometry and the FEM assembly do not need to be rewritten.

```
do jz = 1, naz
  do jy = 1, nay
    xground(jy,jz) = YOUR_HEIGHT(dy(jy),dz(jz))
  enddo
enddo
```

Important: xground must be defined at the same horizontal nodal coordinates stored in dy(:) and dz(:). ygrid and zgrid must remain strictly increasing.

2.2. Analytical terrain

For controlled tests, a smooth hill is convenient, for example a Gaussian hill. Example (recommended test case, not yet the default source):

```
xground(jy, jz) = A*dexp(-((dy(jy)-y0)**2
&                +(dz(jz)-z0)**2)/(2.0d0*L**2))
```

This is a useful next validation case because it provides a smooth surface and avoids the gradient jumps that can occur with a raw raster.

2.3. DEM / raster / surveyed terrain elevations

For production data, the recommended workflow is:

1. Read the DEM or terrain data once before the time-stepping loop.
2. Transform the data coordinates so that y, z and terrain elevation are in metres and match the VANADIS coordinate system.
3. Resample/interpolate the DEM onto the horizontal VANADIS nodes (dy(jy), dz(jz)).
4. Store the result in xground(jy,jz).
5. Only then call siatka to build the full three-dimensional terrain-following mesh.

Vertical datum: If the DEM contains absolute elevations (for example metres above sea level) while dx spans only 0...420 m, use a deliberate common datum/offset. Do not blindly insert, for example, a 350 m terrain elevation into a domain with a 420 m top without defining what the remaining height above ground is meant to represent.

3. What the terrain-following mesh does

For every horizontal (y,z) column, the current code obtains xg and maps all vertical levels between the local ground and the unchanged top boundary using:

$$\begin{aligned}\sigma &= (\text{dx}(j) - \text{xbase}) / (\text{xtop} - \text{xbase}) \\ x &= \text{dx}(j) + (1 - \sigma) * (\text{xg} - \text{xbase})\end{aligned}$$

- $\sigma = 0$: the bottom node lies exactly on $x = \text{xg}(y,z)$.
- $\sigma = 1$: the top boundary remains fixed at $x = \text{xtop}$.
- intermediate layers deform linearly.
- neighbouring elements share global nodes, so no gaps are introduced.

Required condition: $\text{xg} < \text{xtop}$ must hold everywhere. This alone does not guarantee good HEX8 quality; $\det(J)$ and Jacobian conditioning must also be checked.

4. Terrain and the velocity field

4.1. Why constant $V=(0,-5,0)$ is not always appropriate

For a sloping surface $x=\text{xg}(y,z)$, a normal vector is proportional to $n = (1, -\text{dxg}/\text{dy}, -\text{dxg}/\text{dz})$. If the ground is impermeable, $v \cdot n = 0$. Therefore:

$$v_1 = v_2 * (\text{dxg}/\text{dy}) + v_3 * (\text{dxg}/\text{dz})$$

The current VANADIS velocity_field uses exactly this construction. $v(2)$ and $v(3)$ currently come from the configuration. In this terrain-aware version, the configured $V1$ value is ignored and $v(1)$ is constructed from the local terrain slope.

4.2. When the correction should be used

Situation	Recommended action
-----------	--------------------

Synthetic test: v_2, v_3 are prescribed and the ground must be impermeable	Use $v_1 = v_2 \cdot x_g, y + v_3 \cdot x_g, z$. This is the current verified test construction.
Full v_1, v_2, v_3 field from CFD/meteo for the same terrain	Use the field directly if it is already physically consistent with that terrain.
Full measured field but noisy near the ground	Interpolate first and inspect $v \cdot n$. Optionally enforce tangency only at/very near the boundary if impermeability is the intended condition.
Only v_2, v_3 measured near the ground	v_1 may be reconstructed from the slope to impose no-penetration, but this is a model for the missing component, not a measurement.
Only a few measurement stations	This is not enough to assign a velocity at every Gauss point directly. A spatial model/interpolation is required, and time interpolation if the field is transient.

5. Can a measured velocity field be inserted directly?

Yes - this is the preferred route for realistic applications. VANADIS calls `velocity_field(i_czas,x,y,z,v)` locally at Gauss points. That routine can return velocities interpolated from measurements, CFD, a meteorological model or reanalysis data. The assembly does not need to be changed.

The following requirements should be satisfied:

- **Units:** v in m/s, coordinates in m, and time consistent with i_czas/dt .
- **Axis convention:** transform the data to VANADIS coordinates: x vertical, y and z horizontal.
- **Same geometry:** the velocity field must refer to the same terrain/datum used in `xground`.
- **Full domain coverage:** define interpolation inside the dataset and a deliberate policy outside its valid range.
- **No file I/O at Gauss points:** read files once into arrays before time stepping; `velocity_field` should only interpolate in-memory data.
- **OpenMP:** keep the field arrays read-only during assembly so interpolation remains thread-safe.
- **Ground condition:** check $v \cdot n$. If the real field already respects the terrain, do not automatically apply the slope correction.

6. Recommended implementation pattern for measured data

This is a recommended extension. The current source does not yet contain a production reader for velocity data.

```

program VANADIS
...
call terrain_init(...)
call velocity_data_init(...)      ! read once
call siatka(...)
... time loop ...
end

```

Then `velocity_field` should perform interpolation only:

```

subroutine velocity_field(i_czas,x,y,z,v)
...
call velocity_interp(i_czas,x,y,z,v)
return
end

```

For a 4-D dataset, a typical implementation uses spatial interpolation (for example trilinear interpolation on the source grid) plus linear interpolation between the nearest time levels. If the source data use terrain-following levels or height above ground, explicitly convert their vertical coordinate to the physical x coordinate used by VANADIS.

7. Optional no-penetration enforcement for noisy data

If the measured/interpolated velocity at the ground contains a small artificial normal component, while an impermeable ground boundary is intended, the velocity can be projected onto the local tangent plane. This is a recommended generalization, not code currently implemented in VANADIS:

$$\mathbf{n} = (1, -x_g, y, -x_g, z) / |(1, -x_g, y, -x_g, z)|$$

$$\mathbf{v}_t = \mathbf{v} - (\mathbf{v} \cdot \mathbf{n}) \mathbf{n}$$

Do not apply this everywhere: Projection is appropriate at the ground boundary if that physical boundary condition is intended. A real vertical velocity above the terrain may be physically meaningful and should not be removed throughout the three-dimensional domain.

8. Mass balance with a real $\mathbf{v}(x,y,z,t)$ field

VANADIS solves the advective form $\mathbf{v} \cdot \text{grad}(S)$. For a spatially varying velocity field, the physical mass balance contains a $\text{div}(\mathbf{v})$ contribution. The current code evaluates $\text{div}(\mathbf{v})$ numerically with centred differences for mass-balance diagnostics.

Interpretation: The current terrain-consistent test gives $M_{\text{div}} = 0$. A real measured/CFD field does not have to give zero M_{div} . $M_{\text{div}} \neq 0$ is not automatically an error; it must be interpreted together with the physical field and the interpolation procedure.

9. Quality-control checklist after introducing a new terrain

1. **Flat regression.** First set $x_{\text{ground}}=0$ and confirm the flat reference solution.
2. **Geometry.** Check positive $\det(J)$ at every Gauss point and monitor max condF / Jacobian conditioning.
3. **Top boundary.** Verify $x_g < x_{\text{top}}$ everywhere and maintain sensible thickness of the lowest layers.
4. **Normal velocity flux.** Evaluate $\mathbf{v} \cdot \mathbf{n}$ at the ground; for an impermeable ground it should be close to zero.
5. **Mass balance.** Monitor M_{out} , M_{in} , M_{div} , M_P , M_{α} , M_{dir} and eps_{num} .
6. **Nonlinear stability.** Compare $P(S)$ ranges, Picard iterations and DBCG iterations.
7. **Quadrature.** For more strongly warped meshes, compare 8 GP/4 GP with 27 GP/9 GP before assuming the lower-order quadrature is sufficient.

Current regression values for the 12 Sep 2026 code are:

Test	Final result
Flat $x_g=0$, 27 GP + coloring	$M(2000 \text{ s}) = 411.94623624538661 \text{ kg}$
Terrain 0,1,4,7 + terrain-consistent $\mathbf{v}$	$M(2000 \text{ s}) \approx 411.8890214385458 \text{ kg}$; OUT/IN $\sim 1\text{e-}16 \text{ kg/step}$; $M_{\text{div}}=0$

10. Practical decision table

Available data	Recommended approach
Terrain shape + constant horizontal velocity only	Fill x_{ground} and construct $\mathbf{v}_1$ from the terrain slope - a good numerical test.
DEM + full CFD field computed over that DEM	Load the DEM into x_{ground} and interpolate the CFD velocity without artificial slope correction.
DEM + 3-D meteorological model field	Transform coordinates, interpolate $\mathbf{v}(x,y,z,t)$, and check consistency with the surface.
DEM + measurements from multiple masts/stations	Build an interpolated/assimilated field; do not assign a point

	measurement directly to the whole domain.
Near-ground measurements only	A vertical-profile model or additional data are required; the slope correction can only enforce $v \cdot n = 0$ at the ground.

11. What is already implemented and what remains to be added

Already in the current source	Needed for production workflows
xground(jy,jz) + bilinear terrain_geometry	DEM/raster/GIS reader
terrain-following mesh with fixed top	automatic datum handling and DEM resampling
velocity_field(x,y,z,t) API	velocity-file reader and interpolator
terrain-consistent synthetic velocity	$v \cdot n$ QA and optional boundary projection
full mass balance including M_div	automated mesh-quality reports for large DEMs

Summary: For realistic terrain, treat xground as the geometry of the domain and $v(x,y,z,t)$ as independent physical input data. The slope-based correction is excellent for controlled tests and for reconstructing a missing ground-normal condition, but it should not overwrite a valid measured/CFD velocity field.

Document basis: final VANADIS.FOR dated 12 Sep 2026, the verified technical handoff, and the final 0,1,4,7 terrain test. Recommendations concerning production DEM/measurement readers are integration guidance; such readers are not yet present in the current source code.